

Relazione Metodi del Calcolo Scientifico

Secondo Progetto

Marelli Samuele 899768 - Fumagalli Andrea 900329

15 giugno 2026

1 Introduzione

Il linguaggio scelto per il progetto risulta essere Python, famoso linguaggio open-source spesso utilizzato per applicazioni scientifiche. Python offre innumerevoli librerie tra cui scegliere per soddisfare il calcolo delle Trasformate di Fourier; abbiamo deciso di usufruire di diverse tra le più famose:

- **NumPy** una delle librerie più utilizzate per la gestione di array e altricostrutti, utilizzata per gestire i segnali tradotti in array nel nostro caso.
- **SciPy**, in particolare il modulo **fft** che contiene funzioni per calcolare le Trasformate di Fourier, da questo modulo importiamo le funzioni *Fast* per calcolare le trasformate ed eseguire la DCT a 1 e 2 dimensioni. In particolare utilizziamo le funzioni **dct** e **dctn**, entrambe con parametro **norm** = **'ortho'** per rispettare i valori scalati forniti nel testo.
- **Pillow (PIL)** per caricare e convertire i file **.bmp** in matrici della libreria NumPy.
- **Tkinter** per gestire l'interfaccia utente che permette di scegliere l'immagine da comprimere e i parametri di compressione.

La relazione si struttura in 2 capitoli distinti:

- **DCT2 'homemade'** sezione in cui si mostra la nostra implementazione della DCT1 e DCT2, confrontata successivamente con la versione *Fast* della libreria, discutendo i tempi di esecuzione.
- **Compressione Immagini Stile JPEG**: si discuterà della struttura del nostro codice per realizzare la compressione di una immagine in scala di grigi applicando la DCT2, comparando l'immagine compressa a quella originale e discutendo il risultato ottenuto.

2 Implementazione DCT2 'fatta in casa'

2.1 Libreria di supporto utilizzate

Per la realizzazione della DCT1 e DCT2 non sono state usate librerie per il calcolo delle trasformate di Fourier; sono però state sfruttate alcune funzionalità legate al calcolo

matematico e, in particolare, alla gestione degli array grazie rispettivamente alle librerie **Math** e **NumPy**. Di seguito riportiamo la configurazione di queste:

```
1 import math
2 import numpy as np
```

2.2 Codice DCT1

Per l'implementazione della Trasformata Discreta del Coseno (DCT) per matrici quadrate (in 2d) abbiamo deciso iniziare scrivendo l'algoritmo per calcolare la Trasformata per un singolo vettore monodimensionale (1d), successivamente applicheremo questa funzione alle righe e alle colonne della matrice, come mostrato a lezione. Il codice per la DCT1 è il seguente:

```
1 def dct_1d(segnale):
2     N = np.shape(segnale)[0]
3     # Vettore per memorizzare i coefficienti DCT
4     coeff = np.array([0.0] * N)
5     # Ciclo per le frequenze
6     for k in range(N):
7         somma = 0.0
8         # Ciclo dei singoli campioni del segnale
9         for j in range(N):
10             angolo = k * math.pi * (2 * j + 1) / (2 * N)
11             somma += segnale[j] * math.cos(angolo)
12         # Normalizzazione del coefficiente
13         if k == 0:
14             alpha = math.sqrt(1.0 / N)
15         else:
16             alpha = math.sqrt(2.0 / N)
17         coeff[k] = alpha * somma
18     return coeff
```

La funzione utilizza un doppio ciclo for, il primo serve per selezionare la singola frequenza k su cui operare in quella iterazione, il secondo ciclo for invece calcola il coefficiente della frequenza in analisi. Prima di calcolare il coefficiente si stabilisce il fattore di normalizzazione da utilizzare per quella iterazione. Una volta calcolati i coefficienti vengono ritornati sotto forma vettoriale.

2.3 Codice DCT2

Come anticipato, il calcolo della DCT2 homemade si basa sull'applicazione della DCT1 prima alle righe della matrice originale, e successivamente alle colonne della matrice risultante.

Otteniamo il numero di righe e colonne della matrice originale, creiamo una nuova matrice intermedia, inizializzata a zero, basata sulla dimensione di quella originale all'interno della quale verranno immagazzinati i coefficienti della DCT1 applicata sulle righe.

```

1 def dct_2d(matrice):
2     num_righe = np.shape(matrice)[0]
3     num_colonne = np.shape(matrice)[1]
4     matrix_temp = np.zeros((num_righe, num_colonne))
5     # Applica dct1 su ogni riga
6     for i in range(num_righe):
7         matrix_temp[i, :] = dct_1d(np.squeeze(matrice[i, :]))
8     matrix_final = np.zeros((num_righe, num_colonne))
9     # Applica dct1 su ogni colonna
10    for j in range(num_colonne):
11        colonna = np.squeeze(matrix_temp[:, j])
12        matrix_final[:, j] = dct_1d(colonna)
13    return matrix_final

```

Calcoliamo questi coefficienti applicando la DCT1 sulle righe; definiamo la matrice finale, che conterrà i coefficienti corretti, inizializzandola con zeri. La matrice finale sarà riempita con i coefficienti calcolati applicando la DCT1 alle righe della matrice intermedia calcolata al passo precedente. Infine ritorniamo la matrice finale dei coefficienti.

2.4 Validazione algoritmo

Per assicurarci che il calcolo risulti corretto abbiamo confrontato i coefficienti calcolati da entrambe le DCT: sia quella *homemade* che quelle implementate nelle librerie; mettendoli a paragone con quelli calcolati su una matrice nota. Per la correttezza dei coefficienti dell'algoritmo DCT1, utilizziamo il vettore:

$$\begin{bmatrix} 231 & 32 & 233 & 161 & 24 & 71 & 140 & 245 \end{bmatrix}$$

I coefficienti dati dall'applicazione della DCT1 a questo vettore devo essere:

$$\begin{bmatrix} 4.01e+02 & 6.60e+00 & 1.09e+02 & -1.12e+02 & 6.54e+01 & 1.21e+02 & 1.16e+02 & 2.88e+01 \end{bmatrix}$$

Eseguendo gli algoritmi notiamo che:

- la DCT1 homemade ritorna i corretti coefficienti grazie all'impostazione di normalizzazione.
- la libreria invece necessita della specifica `norm = 'ortho'`, per comunicare alla funzione di non utilizzare la normalizzazione di default (backward), ma applicare quella ortonormale.

Per la correttezza della DCT2 invece, utilizziamo la matrice:

$$\begin{bmatrix} 231 & 32 & 233 & 161 & 24 & 71 & 140 & 245 \\ 247 & 40 & 248 & 245 & 124 & 204 & 36 & 107 \\ 234 & 202 & 245 & 167 & 9 & 217 & 239 & 173 \\ 193 & 190 & 100 & 167 & 43 & 180 & 8 & 70 \\ 11 & 24 & 210 & 177 & 81 & 243 & 8 & 112 \\ 97 & 195 & 203 & 47 & 125 & 114 & 165 & 181 \\ 193 & 70 & 174 & 167 & 41 & 30 & 127 & 245 \\ 87 & 149 & 57 & 192 & 65 & 129 & 178 & 228 \end{bmatrix}$$

Questa deve ritornare i coefficienti:

$$\begin{bmatrix} 1.11e+03 & 4.40e+01 & 7.59e+01 & -1.38e+02 & 3.50e+00 & 1.22e+02 & 1.95e+02 & -1.01e+02 \\ 7.71e+01 & 1.14e+02 & -2.18e+01 & 4.13e+01 & 8.77e+00 & 9.90e+01 & 1.38e+02 & 1.09e+01 \\ 4.48e+01 & -6.27e+01 & 1.11e+02 & -7.63e+01 & 1.24e+02 & 9.55e+01 & -3.98e+01 & 5.85e+01 \\ -6.99e+01 & -4.02e+01 & -2.34e+01 & -7.67e+01 & 2.66e+01 & -3.68e+01 & 6.61e+01 & 1.25e+02 \\ -1.09e+02 & -4.33e+01 & -5.55e+01 & 8.17e+00 & 3.02e+01 & -2.86e+01 & 2.44e+00 & -9.41e+01 \\ -5.38e+00 & 5.66e+01 & 1.73e+02 & -3.54e+01 & 3.23e+01 & 3.34e+01 & -5.81e+01 & 1.90e+01 \\ 7.88e+01 & -6.45e+01 & 1.18e+02 & -1.50e+01 & -1.37e+02 & -3.06e+01 & -1.05e+02 & 3.98e+01 \\ 1.97e+01 & -7.81e+01 & 9.72e-01 & -7.23e+01 & -2.15e+01 & 8.13e+01 & 6.37e+01 & 5.90e+00 \end{bmatrix}$$

Analogamente al caso precedente, la DCT1 homemade ritorna i coefficienti corretti per costruzione, mentre quella derivata dalla libreria necessita del parametro `norm = 'ortho'` nella chiamata di funzione.

2.5 Confronto tempi di esecuzione

Come passaggio finale, abbiamo confrontato i tempi di esecuzione della nostra implementazione con la versione *Fast* della libreria **SciPy**. Per valutare le tempistiche abbiamo generato matrici quadrate di dimensione $N \times N$ con N crescente, eseguendo entrambi gli algoritmi e misurando il tempo impiegato per calcolare le trasformate. Di seguito riportiamo una tabella con i vari tempi di esecuzione per i diversi valori di N :

Dimensione	DCT2 Homemade	DCT2 SciPy
$N = 4$	0.000113 s	0.000036 s
$N = 8$	0.000448 s	0.000030 s
$N = 16$	0.003077 s	0.000035 s
$N = 32$	0.019871 s	0.000070 s
$N = 64$	0.140704 s	0.000093 s
$N = 128$	1.199802 s	0.000170 s
$N = 256$	10.125923 s	0.000477 s

Per visualizzare meglio l'andamento asintotico abbiamo inserito i dati su un grafico a scala semilogaritmica sulle ordinate.

Come mostrato dall'andamento del grafico, i tempi riflettono la complessità teorica. La nostra implementazione della DCT2 possiede una crescita cubica, il tempo è quindi proporzionale a N^3 ; l'algoritmo di SciPy invece sfrutta l'ottimizzazione fft (Fast Fourier Transform) con tempi di esecuzione proporzionali a $N^2 \log(N)$, risultando quindi ordini di grandezza più efficiente al crescere di N . Una anomalia segnalata è la discrepanza con valori di N molto piccoli, infatti quando le matrici (e quindi le operazioni) risultano di scarsa dimensione, l'implementazione di **SciPy** tende ad essere irregolare, atteggiando dovuto all'overhead aggiunto dalla chiamata alla libreria (trascurabile sulle grandi dimensioni).

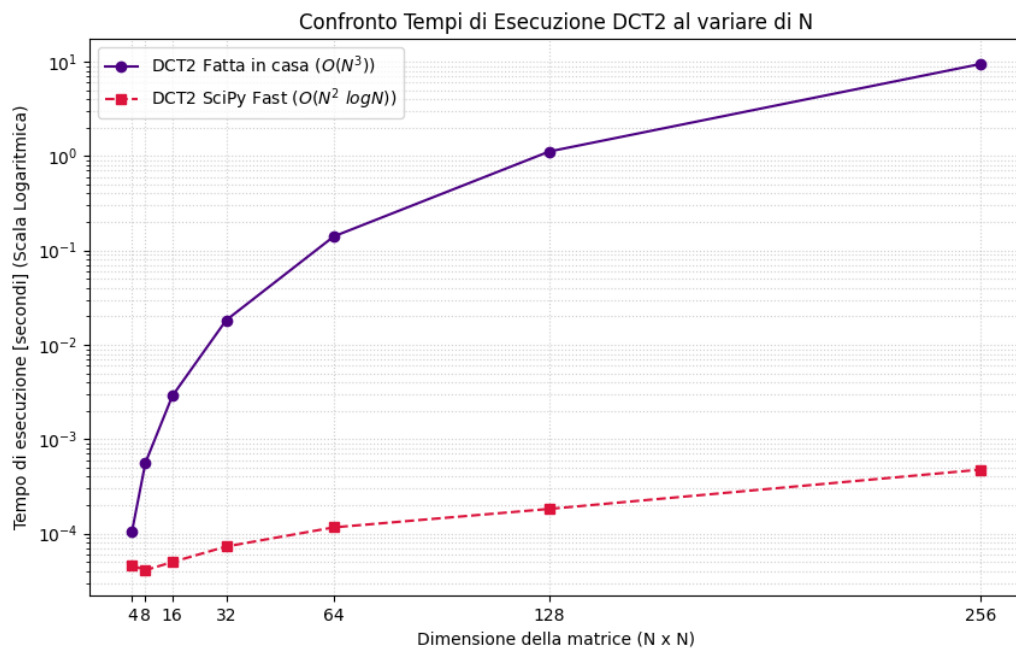


Figura 1: Confronto tempi di esecuzione al variare di N